

ArrayList — 배열 기반 동적 리스트

코딩테스트에서 가장 많이 쓰는 자료구조. 순서가 있고 중복을 허용하는 동적 배열.

계층 위치: `Iterable` → `Collection` → `List` → `ArrayList`

핵심 메서드

메서드	설명	시간복잡도
<code>add(e)</code>	끝에 추가	$O(1)$
<code>add(index, e)</code>	특정 위치에 삽입	$O(n)$
<code>remove(index)</code>	인덱스로 제거	$O(n)$
<code>remove(Object o)</code>	값으로 제거 (첫 번째만)	$O(n)$
<code>get(index)</code>	인덱스로 접근	$O(1)$
<code>set(index, e)</code>	인덱스 위치 값 변경	$O(1)$
<code>size()</code>	크기 반환	$O(1)$
<code>subList(from, to)</code>	부분 리스트 (뷰)	$O(1)$
<code>sort(comparator)</code>	정렬 (원본 변경)	$O(n \log n)$
<code>indexOf(o)</code>	값의 첫 번째 인덱스 (-1 if 없음)	$O(n)$
<code>contains(o)</code>	값 존재 여부	$O(n)$
<code>isEmpty()</code>	비어있는지 확인	$O(1)$

시간복잡도 $O(1)$ 인 메서드가 빠른 이유

- `size()`, `isEmpty()`: 내부에 `size` 필드를 두고 `add/remove` 시마다 갱신해둠. 호출 시 해당 값만 반환
- `get(index)`, `set(index, e)`: 배열 기반이므로 인덱스로 바로 접근 (배열 주소 + offset 계산)
- `subList(from, to)`: 요소를 복사하지 않고, 원본 리스트의 참조 + from/to 인덱스만 저장하는 뷰(view) 객체 생성
- `sort(comparator)`: 내부적으로 TimSort (Merge Sort + Insertion Sort 하이브리드) 사용. 커스텀 Comparator를 넘겨도 비교 기준만 바뀔 뿐 정렬 알고리즘은 동일하므로 $O(n \log n)$

패턴

코딩테스트 출제 빈도: 패턴1 스택처럼 사용 > 패턴2 부분 배열 추출 > 패턴3 존재 여부 + 위치 확인 (JS: 찾아서 제거) > 패턴4 모든 쌍 비교 (JS: 2차원 배열 순회)

패턴 1: 스택처럼 사용 — `add / get(size()-1)`

언제 쓰는가

- 이전 값과 현재 값을 비교해야 할 때 (연속 중복 제거, 괄호 검증 등)
- 마지막에 넣은 값을 꺼내서 비교하는 LIFO 패턴

실무에서는

- Undo/Redo 기능, 브라우저 뒤로가기 버튼

```
import java.util.*;

public class StackLikeUsage {
    public static void main(String[] args) {
        // 연속 중복 제거: [1, 1, 3, 3, 0, 1, 1] → [1, 3, 0, 1]
        int[] arr = {1, 1, 3, 3, 0, 1, 1};
        List<Integer> result = new ArrayList<>();

        for (int num : arr) {
            // 리스트가 비어있거나, 마지막 값과 다르면 추가
            if (result.isEmpty() || !result.get(result.size() - 1).equals(num)) {
                result.add(num);
            }
        }

        System.out.println(result); // [1, 3, 0, 1]
    }
}
```

실제 Stack 클래스를 사용하지 않는 이유

```
// ❌ Stack 클래스 - 레거시, 사용 비권장
// Stack은 Vector를 상속받았고, Vector는 모든 메서드에 synchronized가 걸려있음
// → 단일 스레드에서도 불필요한 동기화 오버헤드 발생
Stack<Integer> stack = new Stack<>();

// ✅ 대안 1: ArrayDeque - Java 공식 권장
// 스택/큐 기능이 모두 필요할 때
Deque<Integer> deque = new ArrayDeque<>();
deque.push(1);           // 스택 push
deque.peek();            // 스택 top 확인
deque.pop();             // 스택 pop

// ✅ 대안 2: ArrayList - 코딩테스트에서 간편하게 사용
// 별도 자료구조 없이 스택 동작만 필요할 때
List<Integer> list = new ArrayList<>();
list.add(1);              // push
list.get(list.size() - 1); // peek
list.remove(list.size() - 1); // pop
```

패턴 2: 부분 배열 추출 — subList

언제 쓰는가

- 특정 범위만 잘라서 정렬하거나 처리해야 할 때
- 슬라이딩 윈도우에서 구간 추출

실무에서는

- **페이지네이션**: 전체 데이터를 일정 단위로 나눠서 보여줌. 게시판 1페이지에 20개씩 표시할 때 `subList(0, 20)`, `subList(20, 40)` 식으로 구간을 잘라서 해당 페이지 데이터만 추출
- **배치 처리**: 대량 데이터를 한꺼번에 처리하지 않고 묶음 단위로 나눠서 처리. 1만 건의 알림을 100건씩 잘라서 순차 전송하는 식 (한꺼번에 처리하면 메모리/서버 부담)

```
import java.util.*;

public class SubListExample {
    public static void main(String[] args) {
        List<Integer> list = new ArrayList<>(Arrays.asList(5, 3, 8, 1, 9, 2, 7));

        // 인덱스 2~4 구간만 추출 (8, 1, 9)
        List<Integer> sub = new ArrayList<>(list.subList(2, 5));
        System.out.println(sub); // [8, 1, 9]

        // 부분 리스트 정렬 후 원본에 반영
        Collections.sort(list.subList(2, 5));
        System.out.println(list); // [5, 3, 1, 8, 9, 2, 7] - 인덱스 2~4만 정렬
    }
}
```

패턴 3: 패턴 3: 존재 여부 + 위치 확인 — contains / indexOf

언제 쓰는가

- 값이 존재하는지 확인하면서 동시에 위치도 필요할 때
- **특정 순서 기반 조건 체크**: 우선순위 목록에서 두 값의 순서를 비교. `indexOf("gold") < indexOf("silver")`이면 gold가 더 높은 순위

실무에서는

- **검색 결과 위치 표시**: 목록에서 특정 항목이 몇 번째에 있는지 `indexOf()`로 찾아서 "3번째 항목에 있습니다" 같은 피드백 제공
- **화이트리스트 체크**: 허용된 값 목록(`[".jpg", ".png", ".gif"]`)에 `contains(".exe")`로 포함 여부를 확인해 허용되지 않은 파일을 차단. 블랙리스트는 반대로 차단 목록에 있는지 확인

```
import java.util.*;

public class ContainsIndexOfExample {
    public static void main(String[] args) {
        List<String> ranking = new ArrayList<>(Arrays.asList("gold",
"silver", "bronze"));

        // 존재 여부 확인
        String medal = "silver";
        if (ranking.contains(medal)) {
            int position = ranking.indexOf(medal);
            System.out.println(medal + "은(는) " + (position + 1) + "등입니
다"); // silver은(는) 2등입니다
        }

        // 없는 값 확인
        System.out.println(ranking.indexOf("platinum")); // -1
    }
}
```

코딩테스트 순위 문제: 정렬 + indexOf vs 정렬 + HashMap

```
import java.util.*;

public class RankingComparison {
    public static void main(String[] args) {
        // 점수 기준 순위를 매기는 경우

        // 방법 1: 정렬 + indexOf - O(n) 조회, 단순하지만 느림
        List<Integer> scores = new ArrayList<>(Arrays.asList(85, 92, 78,
95, 88));
        List<Integer> sorted = new ArrayList<>(scores);
        sorted.sort(Comparator.reverseOrder()); // [95, 92, 88, 85, 78]
        int rank = sorted.indexOf(92) + 1; // 2등 - but indexOf는 매번
O(n)

        // 방법 2: 정렬 + HashMap - O(1) 조회, 데이터 클 때 유리 ✅
        Map<Integer, Integer> rankMap = new HashMap<>();
        for (int i = 0; i < sorted.size(); i++) {
            rankMap.put(sorted.get(i), i + 1); // {95=1, 92=2, 88=3,
85=4, 78=5}
        }
        int fastRank = rankMap.get(92); // 2등 - O(1)
    }
}
```

패턴 4: 모든 쌍 비교 — 이중 반복문

언제 쓰는가

- 모든 조합(쌍)을 비교해야 할 때 ($O(n^2)$ 허용되는 경우)
- 두 수의 합, 가장 가까운 쌍 찾기 등

실무에서는

- 중복 데이터 감지, 유사도 비교

```
import java.util.*;

public class PairComparison {
    public static void main(String[] args) {
        // 두 수의 합이 target인 쌍 찾기
        List<Integer> nums = Arrays.asList(2, 7, 11, 15);
        int target = 9;

        for (int i = 0; i < nums.size(); i++) {
            for (int j = i + 1; j < nums.size(); j++) {
                if (nums.get(i) + nums.get(j) == target) {
                    System.out.println(nums.get(i) + " + " + nums.get(j) +
                        " = " + target);
                    // 2 + 7 = 9
                }
            }
        }
    }
}
```

주의사항

subList는 뷰(view)를 반환한다 — 복사가 아님

```
List<Integer> list = new ArrayList<>(Arrays.asList(1, 2, 3, 4, 5));
List<Integer> sub = list.subList(1, 3); // [2, 3] - 원본의 뷰
list.set(1, 99);
System.out.println(sub); // [99, 3] - 원본 변경이 뷰에 반영됨!

// 독립적인 복사본이 필요하면:
List<Integer> copy = new ArrayList<>(list.subList(1, 3));
```

remove(int index) vs remove(Object o) 혼동

```
List<Integer> list = new ArrayList<>(Arrays.asList(10, 20, 30));
list.remove(1); // 인덱스 1의 요소 제거 → [10, 30]
```

```
list = new ArrayList<>(Arrays.asList(10, 20, 30));  
list.remove(Integer.valueOf(20)); // 값 20 제거 → [10, 30]  
// Integer 리스트에서 값으로 제거하려면 반드시 Integer.valueOf() 사용
```

인덱스와 값이 같을 때: int가 항상 우선

```
List<Integer> list = new ArrayList<>(Arrays.asList(1, 2, 3));  
list.remove(2);  
// → 값 2를 제거? ❌  
// → 인덱스 2의 요소(값 3)를 제거 ✅ → [1, 2]  
// Java는 int와 Integer 중 int를 우선 매칭하므로 항상 인덱스로 해석됨  
  
// 값 2를 제거하고 싶으면:  
list.remove(Integer.valueOf(2)); // → [1, 3]
```

ArrayList vs LinkedList 선택 기준

- **ArrayList**: 인덱스 접근이 많으면 사용 (get/set O(1)). 코딩테스트에서는 거의 항상 ArrayList.
- **LinkedList**: 맨 앞/중간 삽입·삭제가 빈번하면 고려 (하지만 실제로는 ArrayList가 캐시 효율이 좋아서 대부분 더 빠름).